



Project Name: City Mind - Urban Intelligence System

Course: Artificial Intelligence

**Group Members: Zarrar Nadeem (24i-0735), Shahmeer Zubair (24i-0580),
Raahim Babar(24i-0561)**

Phase 1 : Problem Analysis and Design

1. Introduction

CityMind is an AI-based system that is designed to help cities manage urban challenges more effectively. The city is represented as a grid with nodes, therefore taking the shape of the graph data structure. Where each node represents either a hospital, school, residential area and each node is connected through roads simulating edges. Each node stores its respective information such as, **location type**, **population density**, **risk index**, **accessibility flag** that tells the system whether that location is currently reachable or not. The system is built on five main challenges:

1. Planning the city layout
2. Optimization of road networks
3. Placement of Ambulances
4. Routing emergency teams
5. Predicting crime rates

Each challenge uses a different algorithmic approach, but each component works together to develop a comprehensive city management system. As all five modules share the same city graph, which means that if a change occurs in the city, for example a road gets flooded, every other module receives this feedback and adjusts accordingly.

This highlights the goal of CityMind, as it is not solving each problem in isolation, but rather to combine each working solution into one working system that can simulate real time urban changes.

2. Challenges

2.1 - Challenge: City layout planning

The initial and most crucial step of the project involves starting with an entirely empty grid that represents the city. The objective is to populate this grid with various buildings subject to strict constraints and placement rules; random placement is not an option. In addition to adhering to these special rules for each building type, the system requires mathematical validation to confirm whether a specific layout design is feasible. If a design cannot be implemented, the system must be able to trace and identify the exact rule causing the conflict that leads to the impossible solution.

The AI technique / algorithm:

The constraint heavy nature of this challenge naturally models it as a **Constraint Satisfaction Problem (CSP)**. This approach utilizes heuristics such as **Minimum Remaining Values (MRV)** and **Least Constraining Value (LCV)**, paired with a backtracking algorithm. Additionally, techniques like forward checking or AC-3 (*arc consistency*) are employed to further optimize the search space.

Why did we choose this algorithm?

Since the problem revolves heavily around strict constraints and rules, CSP is the most natural and effective algorithm. It systematically searches through all constraints to find a viable solution, utilizing backtracking to guarantee a valid grid placement if one exists. Integrating MRV and LCV heuristics significantly enhances efficiency by assisting the algorithm in determining which variable or value to evaluate next, thereby reducing overall computation time. Furthermore, applying forward checking or AC-3 minimizes unnecessary backtracks, further increasing the algorithm's performance.

What alternates did we consider?

Because the problem strictly requires constraint satisfaction, other standard algorithms do not fulfill the requirements as effectively as CSP. A Genetic Algorithm was briefly considered but immediately dismissed. While Genetic Algorithms excel at finding a "good enough" approximation, CSP provides a strict, perfect solution by verifying every possibility before confirming a valid placement. For city layout constraints, perfection and rule adherence were prioritized over approximation.

2.2 - Challenge: Road Network Optimization

The goal of this challenge is to construct a road network that connects every location within the city at the minimum possible cost. However, a critical restriction is the requirement for two completely separate, independent paths connecting the primary hospital and the ambulance depot. This ensures that if one road becomes inaccessible, such as in the event of flooding, a fully functioning backup route remains available.

The AI technique / algorithm:

As a minimization problem based on a grid graph, a Minimum Spanning Tree (MST), specifically Kruskal's algorithm, is utilized to minimize the overall road construction costs between buildings, with the costs acting as edge weights. To satisfy the constraint of having two independent emergency paths, the A* search algorithm is employed to find the shortest and most cost effective routes between the hospital and the ambulance depot.

Why did we choose these algorithms?

MST was selected because it guarantees the cheapest path connecting all nodes without creating unnecessary loops. The A* algorithm is specifically suited for generating the required redundant paths. The process works sequentially: first, the MST creates one valid path between the hospital and the ambulance depot. To establish the backup, the A* algorithm traces that initial route, temporarily removes those edges from memory, and runs a second time. This forces the discovery of a completely separate alternative route. This newly found path is then added to the initial MST routes, ensuring a reliable backup is always available.

What alternates did we consider?

The algorithm of *Simulated Annealing* was strongly considered for this problem. This is a local search optimization algorithm. Sometimes, it accepts worse states in the early time to escape the local optima so it can find a cheaper network globally. This alternative was ultimately rejected and the MST combined with A* approach was selected instead. The main reason for this rejection is because of the speed and the determinism. A local search relies on many thousands of random mutations to reach an optimal state, which has the possibility to be very slow during the real-time recalculations. On the other hand, the MST algorithm provides a mathematically guaranteed minimum cost for the base network instantly.

2.3 - Challenge: Ambulance Placement

The large scale of the city grid results in a massive number of possible placement combinations for emergency services. The primary difficulty lies in the complexity of determining optimal

locations for three ambulances across this expansive area. The objective is to place these ambulances while minimizing the response time for a worst-case scenario, ensuring that the individual furthest away from any given ambulance still experiences the shortest possible wait time.

The AI technique / algorithm:

The Genetic Algorithm (GA) is utilized for this challenge. As an optimization search technique inspired by biological evolution, the positions of the ambulances are represented as a chromosome. The algorithm produces populations of random placements, and the fitness of these populations is continuously evaluated. In this scenario, fitness is defined by how low the worst case response time is. The best placements (*strongest chromosomes*) are combined and mutated over multiple generations, progressively creating better solutions to find a near optimal layout without having to check every single possibility in the grid.

Why did we choose this method?

This algorithm is highly efficient for massive search spaces. Checking all possible ambulance placements individually while calculating the worst case scenario response time for each adds significant computational complexity. The GA handles this effectively through the evolutionary process. By retaining the strongest chromosomes and introducing continuous mutation, the algorithm rapidly converges on near perfect solutions, making it highly effective compared to exhaustive iteration.

What alternates did we consider?

The primary alternative considered was a brute force approach, which would involve calculating and evaluating every single possible placement combination one by one to find the absolute maximum optimization. This approach was rejected because it would consume excessive time, resources, and computing power, making it entirely unsuitable for a real time simulation. The Genetic Algorithm was chosen instead because it delivers a near optimal solution in a fraction of the time required by a brute-force method.

2.4 - Challenge: Emergency Routing Under Changing Conditions

Problem Understanding:

The objective of this is to be able to find the shortest path for a medical team, navigating through the city to help trapped civilians. The difficulty is not in finding the shortest path, the difficulty arises due to the dynamics of the city. As there can be possibilities where a road is blocked or flooded, so therefore the system must be able to recalculate an optimal path and that new path must be the next shortest path.

The AI technique / Algorithm:

The chosen algorithm for this challenge is A* Search, combined with graph recalculation triggered by real-time road blockage events. A* is an informed search algorithm that finds the shortest path between two nodes and by using heuristic to guide the search. The heuristic implemented will use Manhattan distance to get an estimate of the remaining distance based on its current location. To make sure the algorithm never over estimates we add the admissibility check. If there is a road that is blocked, the A* algorithm will begin again from the team's position to find a new solution/path. This process continues until all civilians have been reached.

Why did we choose this method?

A* was selected because it is the only search algorithm that combines both correctness and efficiency for this type of problem. It guarantees the shortest path through the admissibility of its heuristic, which satisfies the project's requirement. Unlike the Dijkstra algorithm, which explores all directions without any guidance which is significantly slower.

What Alternatives were Considered?

The Dijkstra Algorithm was the alternative that was considered. It does guarantee the shortest path and is a really good algorithm to traverse graphs. However, as aforementioned, it traverses across all nodes which is significantly slower, and since this city model implemented real time challenges, meaning the algorithm will have to reroute many times, and Dijkstra will be significantly slower. Therefore, A* was the best choice algorithm.

2.5 - Challenge: Crime Risk Prediction and Integration

Problem Understanding:

This challenge requires the system to analyze neighborhood data, and predict crime rate levels as Low, Medium, or High for each node in the city. This prediction is then used as a cost multiplier which in turn means that if the crime rate is high, then the travel cost for both routing and ambulance placement modules increases. The challenge demands to initially group neighborhoods without any labels, and then secondly train a model to classify risk levels based on generated data.

The AI technique / Algorithm:

The first stage uses K-Means clustering, which is an unsupervised algorithm. This will group the city's neighborhoods based on their population density, and proximity to industrial areas. As there is no labelled data, the algorithm finds natural grouping on its own by finding the minimum distance between data points and assigned cluster areas.

The second stage involves generating a crime data set, using the properties of each node already placed in the shared city graph. A logical formula or connection is made using each node's data to assign crime rates to each neighborhood. For example, an area where schools are not present or are very few can be assigned a High level of crime rates.

Why are two different types of learning used?

The first type of learning is where the system detects the trends and patterns using the data that is not labelled to train the system on how to detect patterns and generate data itself. The second step is supervised because once the data set is generated with assigned crime rates a model can be trained to generalize from them.

What alternatives can be used?

Other possible algorithms are well beyond our scope and require a lot of training and time. Algorithms like *DBSCAN*, mentioned with the help of AI. Therefore in our project, the K-means clustering algorithm is the perfect choice.

3. City Model

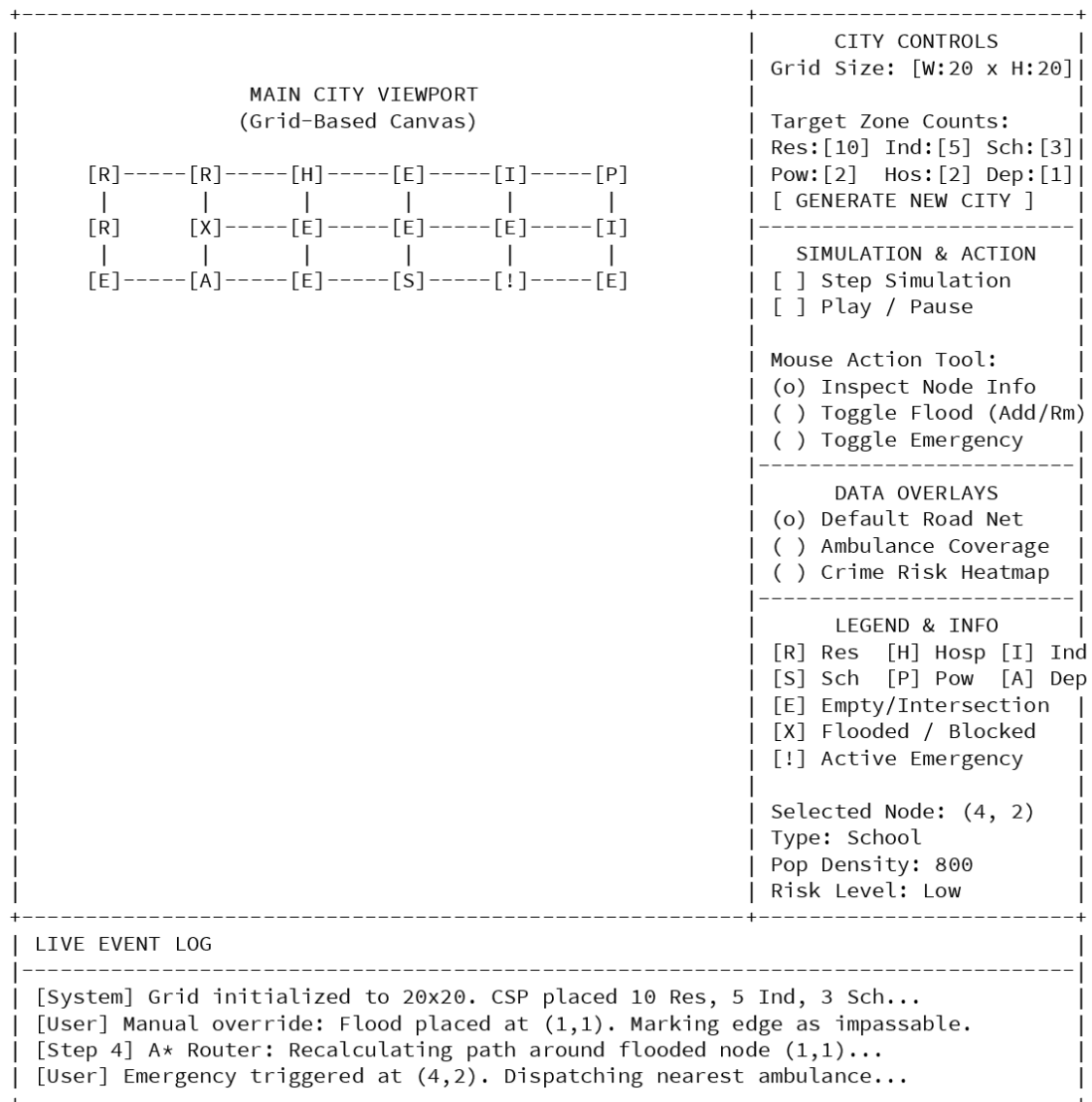
Now that the five challenges have been discussed, a deeper dive can be taken into the actual structure and setup of the city. As aforementioned, the city is represented as a graph made up of nodes and edges. Each node represents a specific location on the map, and every node stores its respective information. Those being **location type** (e.g Hospital, Residential or Industrial). There can also be empty nodes that simply exist as intersections or connecting pathways. **Population Density** which represents the people living in or using the block. **Risk Index** which will be decided by the machine learning model and **Accessibility Flag** which is a boolean that represents if the node can be entered.

Moreover the graph will be in a lattice format to simulate a grid. This means that any node will only be connected to another node in the 4 cardinal directions around it. This type of format will be implemented through an adjacency list where every node will keep a list of its connected neighbors as well as the edge cost required to reach the neighbor.

The edges between nodes are represented as roads, each road has a travel cost associated with it. Standard roads have a cost value of 1, while roads passing through residential areas have a reduced cost of 0.8. The grid size and number of buildings will be decided by the user at runtime with max thresholds in place to keep computing time satisfactory. It should be noted that this graph is a shared object passed by reference to all five challenge modules. No algorithm maintains a separate copy of the city; therefore, an edge cost updated by a flood during the simulation is immediately visible to the routing and ambulance modules and removed from consideration when path finding.

4. Visual Implementation

To ensure the simulation is easy to watch and interact with, the interface will feature a top-down 2D map (akin to old videogames). Graphical sprites will be used for the buildings and roads to make the GUI look more appealing. A rough sketch of the GUI can be pictured below along with an indepth explanation.



4.1 - Visual Rendering System (The Map)

- **Buildings:** Each grid cell will display a specific sprite corresponding to its zoning type (e.g., a house graphic for *Residential* zones, a factory for *Industrial* zones).
- **Roads:** Edges between nodes will be rendered as roads with empty nodes that have multiple roads connected to them being rendered as intersections.
- **Dynamic Entities:** The medical team will be represented by an ambulance sprite that actively transitions between grid cells during the simulation.

4.2 - User Interaction

- **City Setup:** Text boxes to set up grid size as well as number of various types of buildings for creating specific city layouts.
- **Creating a Flood:** When the Flood tool is activated, the system does not require the user to click the road itself. Instead, the user selects a building, followed by an adjacent building. The system automatically identifies the connecting road, severs the connection within the code, and renders a "Water/Flood" graphic exactly between the two structures.
- **Inspecting Details:** A single click on any building populates a side panel with the location's statistics, such as population density and predicted crime risk.
- **Emergency Toggle:** This can be used to click on any node and start an emergency there that the medical team has to respond to.
- **Simulation Controls:** Play/Pause and step-by-step buttons to control the simulation.

4.3 - Data Overlays

To fulfill the project's multiple map view requirements without cluttering the interface, filters can be toggled over the primary map:

- **Road Network View:** Applies color coding to the roads to visually distinguish active routes, discounted residential paths, and flooded sections.
- **Ambulance Coverage:** Colors nodes based on how many edges/cost it takes to get to any node.

- **Crime Risk Heatmap:** Projects a color gradient (e.g., red, yellow, green) across the city grid, visualizing the areas most susceptible to crime based on the Machine Learning module's predictions.

4.4 - The Live Event Log

A text interface at the bottom of the screen will record system states and user actions chronologically.

- **Event Tracking:** If a flood is initiated manually, the log registers the exact coordinates and action. When the routing algorithm dynamically adjusts to that flood, the system immediately logs the recalculation.

5 - Work Breakdown

Zarrar Nadeem (24I-0735):

1. Challenge 5: clustering the neighborhoods, generating the crime dataset, training the classifier, and feeding the risk scores back into the graph.
2. Challenge 4: Handle real-time emergency routing using A* and make it recalculate whenever a road gets blocked
3. Building the live event log that records what happens at each step throughout the simulation.

Shahmeer Zubair (24I-0580):

1. Challenge 3: Figure out the best ambulance positions using the Genetic Algorithm
2. Build the simulation loop that runs all 20 steps and triggers random events like road floods
3. Building the visual interface, showing the grid with all the overlays, including the heatmap

Raahim Babar (24I-0561):

1. Build the shared city graph which is the base structure that every other module runs on

2. Challenge 1: Place all the buildings on the grid following the placement rules using CSP
3. Challenge 2: Build the road network using Kruskal's MST and add the backup route between the hospital and depot using A*